

EXPERIMENT 7

Shravanya Andhale
D20A-02

AIM: Introduction to Truffle, Establish local blockchain using Truffle and deployment of Smart Contract

THEORY:

1. Ganache

Ganache is a personal/local Ethereum blockchain used for **development, testing and experimentation**.

It provides a controlled blockchain environment where developers can:

- Create test accounts.
- Obtain pre-funded test Ether.
- Deploy smart contracts.
- Execute transactions.
- Mine blocks.
- View transaction history.
- Monitor blockchain activity.

Ganache is useful because smart contracts can be developed and tested without using a public Ethereum network or real Ether. The experiment specifically identifies instant mining, pre-funded accounts, transaction history and a graphical interface as Ganache features.

2. MetaMask

MetaMask is a blockchain wallet used to manage Ethereum-compatible accounts and interact with decentralized applications.

In this experiment, MetaMask connects the Ganache accounts to Remix IDE.

The Ganache private key is imported into MetaMask, allowing the corresponding Ganache account to be used for transactions from Remix.

3. Remix IDE

Remix IDE is a browser-based development environment for Solidity smart contracts.

It allows us to:

- Write Solidity programs.

- Compile contracts.
- Deploy contracts.
- Execute transactions.
- Call contract functions.
- Inspect transaction results.

4. Local Blockchain

A blockchain is a distributed ledger in which transactions are recorded in blocks.

For this experiment, instead of using Ethereum Mainnet, we use **Ganache as a local blockchain**.

This allows us to perform the complete blockchain workflow:

account → transaction → block → smart contract state change

without using real cryptocurrency.

5. Smart Contract

A smart contract is a program deployed on a blockchain that contains predefined rules and functions.

For this experiment, we use a smart contract based on the selected mini-project:

Blockchain-Based College Event Certificate Issuance & Verification System.

The contract represents the **blockchain layer** of the certificate system.

It records important certificate information on the blockchain and provides functions for issuing, retrieving, verifying and revoking certificates.

6. Certificate Record

A certificate can be represented using a Solidity struct.

For example:

```
struct Certificate {
    uint256 certificateId;
    string studentName;
    string eventName;
    string certificateCID;
    bytes32 certificateHash;
    address issuer;
    bool revoked;}

```

The record contains:

Field	Purpose
certificateId	Unique certificate identifier
studentName	Name of certificate recipient
eventName	Event for which certificate was issued
certificateCID	IPFS identifier of certificate
certificateHash	Cryptographic proof of certificate
issuer	Address that issued certificate
revoked	Whether certificate has been revoked

7. Mapping

A mapping associates one value with another.

For example:

mapping(uint256 => Certificate) public certificates;

Here:

- uint256 = certificate ID
- Certificate = certificate record

Therefore, a certificate can be retrieved using its unique ID.

For example:

Certificate ID 1001 → Certificate record

Mappings are particularly suitable here because each certificate has a unique ID.

8. msg.sender

msg.sender represents the Ethereum address that called the current function.

When the authority issues a certificate:

issuer: msg.sender

records the blockchain address that performed the issuance.

This provides an on-chain record of who issued the certificate.

9. require()

The require() statement checks whether a condition is satisfied.

If the condition is false, the transaction is reverted.

For example:

```
require(  
    certificates[_certificateId].certificateId == 0,  
    "Certificate already exists"  
);
```

This prevents the same certificate ID from being issued twice.

Another example:

```
require(  
    certificates[_certificateId].certificateId != 0,  
    "Certificate does not exist"  
);
```

ensures that verification is performed only for an existing certificate.

10. Events

Solidity events record important contract activities in transaction logs.

For example:

```
event CertificateIssued(  
    uint256 certificateId,  
    string studentName  
);
```

An event can be emitted whenever a certificate is issued.

Similarly, we can have:

CertificateIssued
CertificateRevoked events.

Events are useful for applications that need to monitor blockchain activity.

11. IPFS and Certificate CID

In the actual project, certificate PDFs are stored off-chain using IPFS.

IPFS returns a **CID (Content Identifier)**.

The CID can then be stored in the smart contract.

12. Certificate Hash

A cryptographic hash can act as a fingerprint of a certificate.

If the certificate contents are changed, its hash changes.

Therefore, storing a certificate hash on-chain allows the system to detect whether the certificate corresponds to the original recorded proof.

13. Certificate Revocation

A certificate may sometimes be issued incorrectly.

Therefore, our contract includes a revoked status.

Initially:

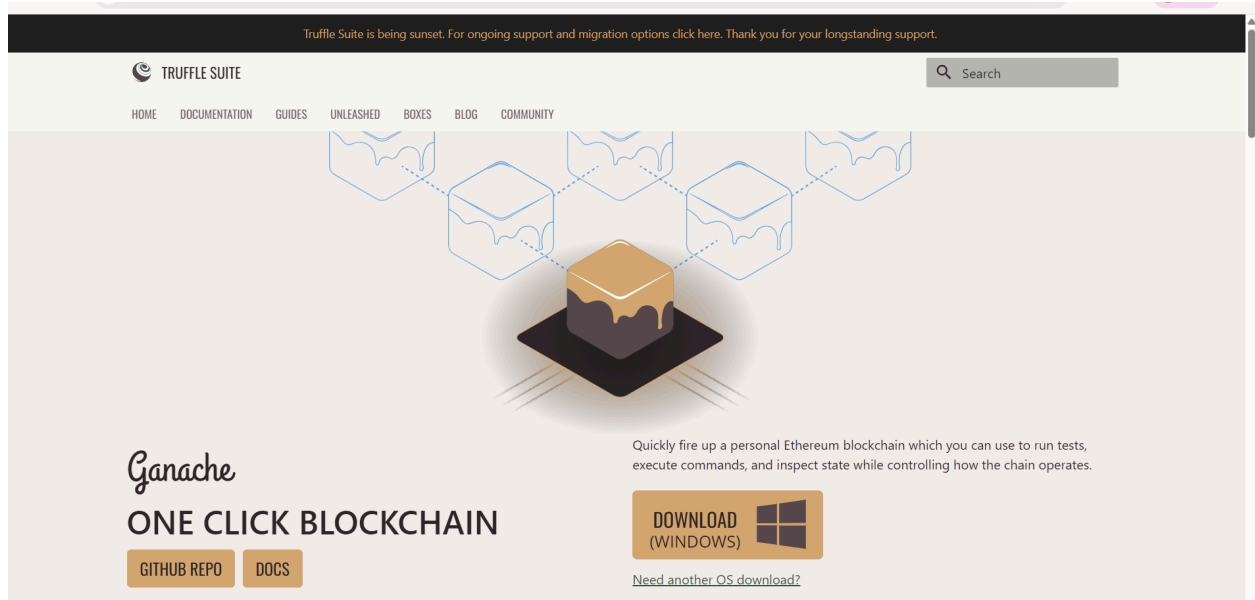
revoked = false

After the authority revokes it:

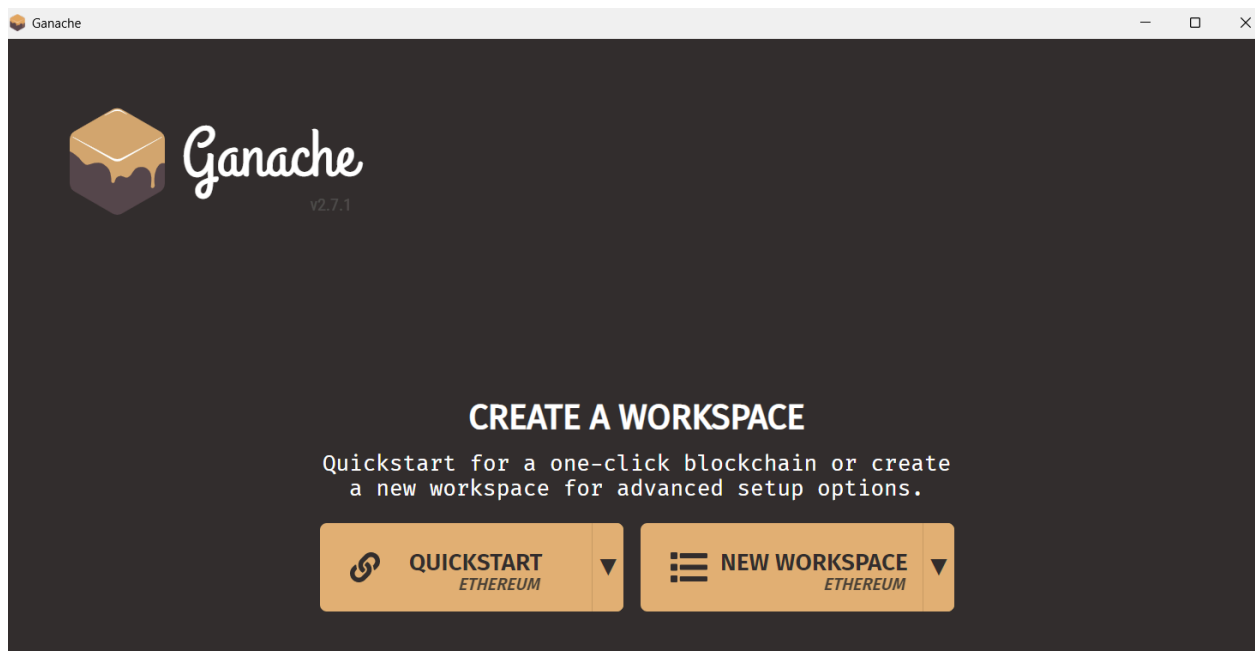
revoked = true

The certificate can then be considered invalid/revoked during verification.

CODE AND OUTPUT:



Using QuickStart Ethereum:



ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

CURRENT BLOCK0

GAS PRICE20000000000

GAS LIMIT6721975

HARDFORKMERGE

NETWORK ID5777

RPC SERVERHTTP://127.0.0.1:7545

MINING STATUSAUTOMINING

WORKSPACE QUICKSTART

SAVE

SWITCH

MNEMONIC ?

HD PATH

siren equip prevent enjoy figure old balcony session absurd orbit sweet hobby

m44'60'0'0account_index

ADDRESS	BALANCE	TX COUNT	INDEX	
0xbc4bB11091d5b7cF20541A6AC37ef8Dc4E39670b	100.00 ETH	0	0	
0x9AE005a6D94FC43E5eeD1C0EB8d81ca07D342D4d	100.00 ETH	0	1	
0xC74A302eCfc743A7A813110DE4162A8b84ced460	100.00 ETH	0	2	
0xEA8c4163dDEC2ced29D48adF0415868856D90b32	100.00 ETH	0	3	
0x7b329Af34E5A8C5B34ac901840c106eaED89FDda	100.00 ETH	0	4	
0x6E7889A5cCBBE05936b81FDfF98101181434aA6D	100.00 ETH	0	5	
0xbb8adfde459a810712dD008c6ea70C36b8ec45ED	100.00 ETH	0	6	

Restarting the Quickstart workspace resets the blockchain. All transactions and contract states will be reset.

SERVER

HOSTNAME

127.0.0.1 - Loopback Pseudo-Interface 1

The server will accept RPC connections on the following host and port.

PORT NUMBER

7545

NETWORK ID

5777

Internal blockchain identifier of Ganache server.

AUTOMINE

☒

Process transactions instantaneously.

ERROR ON TRANSACTION FAILURE

☐

When transactions fail, throw an error. If disabled, transaction failures will only be detectable via the "status" flag in the transaction receipt. Disabling this feature will make Ganache handle transaction failures like other Ethereum clients.

CHAIN FORKING

☐

Fork an existing chain creating a new sandbox with the existing chain's accounts, contracts, transactions and data.

Connecting MetaMask Wallet to Ganache:

MetaMask

<

Add network

×

⚡ Add from ChainList

Network name

Ganache

According to our records, the network name may not correctly match this chain ID.
Suggested name: [Localhost 8545](#)

Default RPC URL

127.0.0.1:7545

▼

Chain ID

1337

Currency symbol

ETH

Block explorer URL

Add a URL

▼

Save

MetaMask

<

Add wallet

⌵

Private key

Imported private keys are backed up to your account and sync automatically when you sign in with the same Google or Apple login.
[Learn more](#) about how imported keys work

Select type

Private key ▼

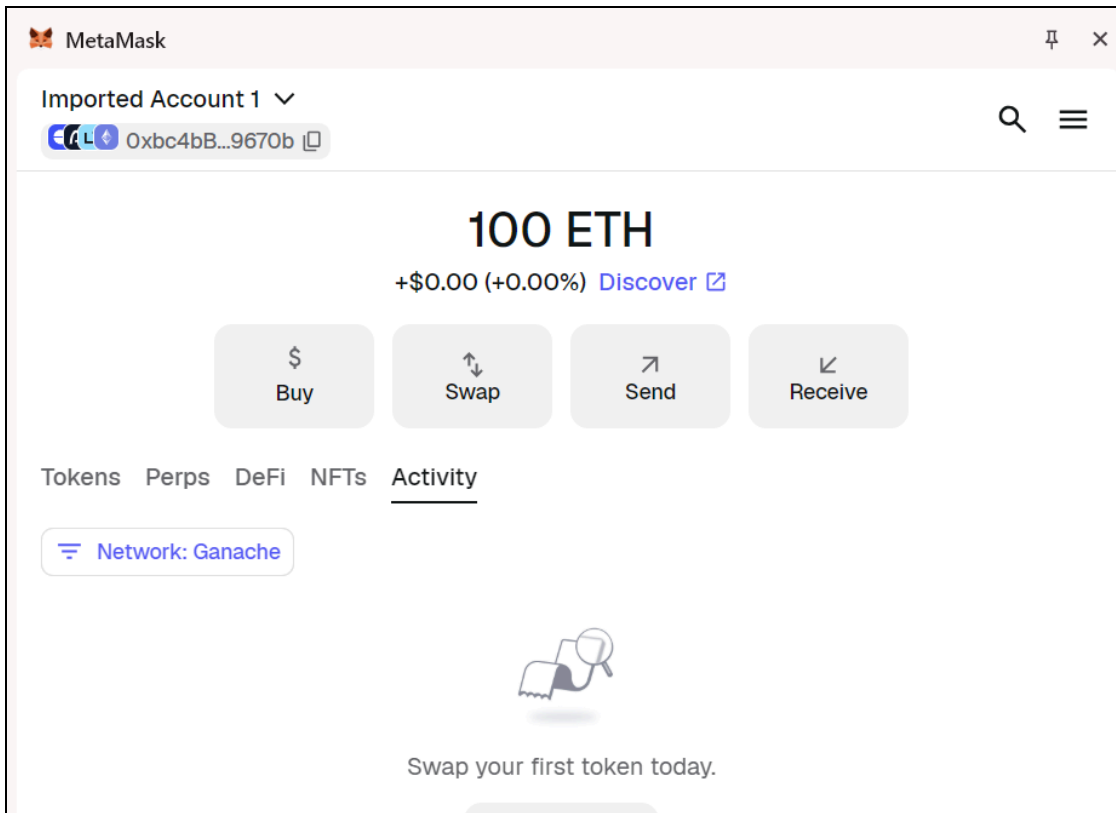
Enter your private key string here:

.....

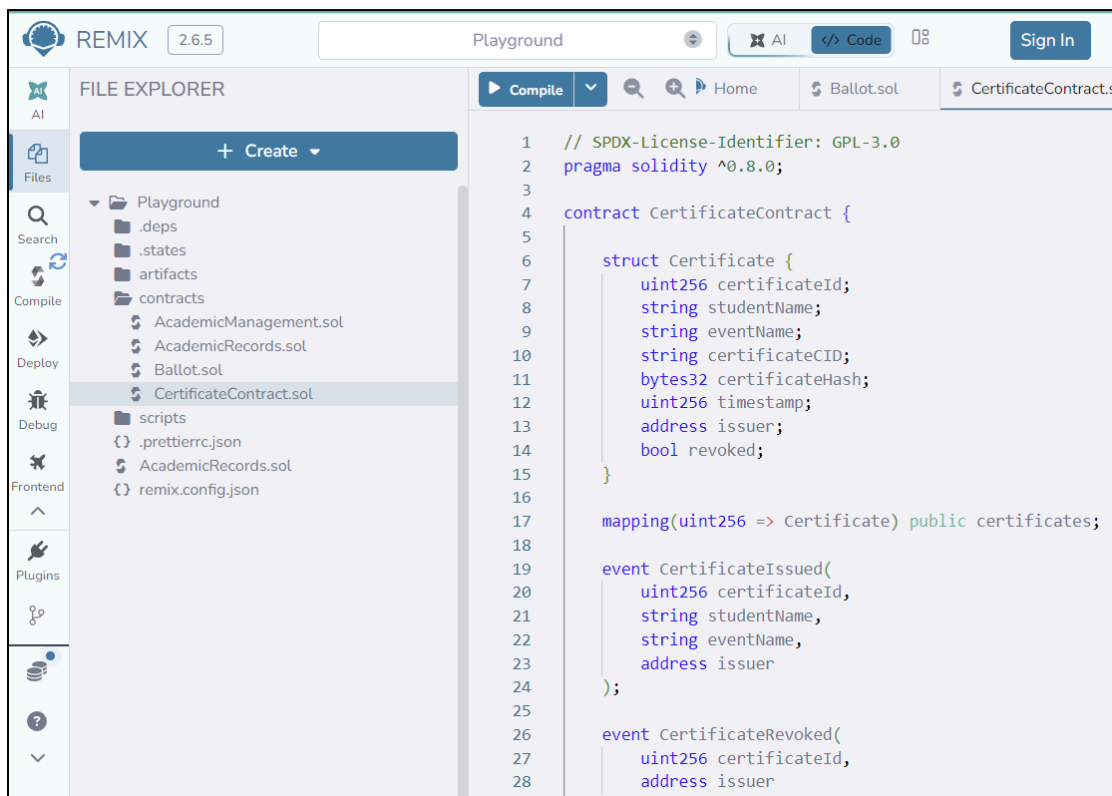
⌵

Cancel

Import



Smart Contract for Certificate Management:



Creating, retrieving, verifying Certificate 1:

REMIX 2.6.5

Playground

AI

Code

08

Sign In

DEPLOY & RUN TRANSACTIONS

Environment

Account 16 0x41a...ed138 0.046 ETH

Deploy

Deployed Contracts

Transaction His

Deploy

CertificateContract

Verify Contract on Explorers

Value 0 wei

Gas limit auto 0

Deploy

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.0;
3
4 contract CertificateContract {
5
6     struct Certificate {
7         uint256 certificateId;
8         string studentName;
9         string eventName;
10        string certificateCID;
11        bytes32 certificateHash;
12        uint256 timestamp;
13        address issuer;
14        bool revoked;
15    }
16
17    mapping(uint256 => Certificate) public certificates;
18
19    event CertificateIssued(
20        uint256 certificateId,
21        string studentName,
22        string eventName,
23        address issuer
24    );
25
26    event CertificateRevoked(
27        uint256 certificateId,
28        address issuer
29    );
30 }
```

Account 1

Wallet 1

Deploy a contract

This site wants you to deploy a contract

Estimated changes No changes

Network s Sepolia

Request from app.remix.live

Signing with Account 1 Wallet 1

Network fee 0.0204 SepoliaETH

Speed Market ~12 sec

Cancel Confirm

Deployed Contracts 1 + Add Contract

Interact with a deployed contract

CertificateContract Sepolia 0xd8f...40B17 6s ago

Functions

Search functions...

issueCe... (uint256, string, string,...)

_certificateId (uint256)

_studentName (string)

eventName (string)

[block:11857669 txIndex:13] from: 0x41a...eD138 to: CertificateContract.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x552...c9f74

status 1 Transaction mined and execution completed

transaction hash 0xf177969161d9aff105ebbcbe67d817aa2e9ea906128901d4dfb7619d6cad7cc9

block hash 0x552da46bfd577ca6b1e62e349e9e4126a3de092790a0ffbd955bb73c6d6c9f74

block number 11857669

contract address 0xd8ff9f758a5b7CA1DDf9247A2A90D173C7540B17

from 0x41af7CCA56dbf8f51ff4192441966267eD138

```

● issueCe... (uint256, string, string,...
  01
  SHRAVANYA ANDHALE
  GENESIS HACKATHON
  QmShravanyaCertificate
  0x11111111111111111111111111111111

```

SHRAVANYA ANDHALE

GENESIS HACKATHON

QmShravanyaCertificate

```
0x11111111111111111111111111111111
```

 MetaMask



Account 1
Wallet 1



Transaction request

Balance changes 

No changes

 Added protection  Optional



Because you're interacting with an unknown address, protection can prevent some malicious transactions. [Learn more](#)

Network

s Sepolia

Request from 

app.remix.live

Interacting with 

 0xd8ff9...40B17

Signing with 

 Account 1
Wallet 1

Network fee 

 0.0029 s SepoliaETH

Speed

Market ~12 sec

Cancel

Confirm

getCertificate (uint256)Call

02

Calldata Params

Auto-Fill with AI Samples

verifyCertificate (uint256)Call

02

Calldata Params

Auto-Fill with AI Samples

decoded input

```
{
  "uint256_certificateId": "2"
}
```

decoded output

```
{
  "0": "bool: true"
}
```

logs

```
[]
```

raw logs

```
[]
```

Revoking Certificate 2:

MetaMask

Account 1

Wallet 1

Transaction request

Balance changes ?

No changes

Added protection ? Optional

Because you're interacting with an unknown address, protection can prevent some malicious transactions. [Learn more](#)

Network

s Sepolia

Request from ?

app.remix.live

Interacting with ?

Oxd8ff9...40B17

Signing with ?

Account 1

Wallet 1

Network fee ?

0.0008

s SepoliaETH

Speed

Market ~12 sec

Cancel

Confirm

CONCLUSION:

Ganache was successfully used as a local Ethereum blockchain for testing and deploying a Solidity smart contract. MetaMask was connected with the Ganache accounts and Remix IDE was connected through the injected provider. A Certificate Issuance and Verification smart contract based on the selected mini-project was successfully compiled, deployed and interacted with. Certificate records were issued, retrieved, verified and revoked through blockchain transactions. The corresponding transactions and blockchain activity were also verified using Ganache.